

Node.js is an **open-source, cross-platform JavaScript runtime environment** that allows you to execute JavaScript code **outside a web browser**. It's built on **Google Chrome's V8 JavaScript engine**, making it highly efficient for building scalable server-side and networking applications.

1. Core Concepts and Architecture

Node.js is defined by its architectural approach, which prioritizes performance for I/O-intensive tasks.

A. Non-blocking I/O and Asynchronicity

Traditional server models often use blocking I/O, where a process waits for a task (like a database query) to finish before moving to the next one. Node.js uses **non-blocking, asynchronous I/O**.

- **Non-blocking:** The process immediately moves to the next task without waiting for the I/O operation to complete.
- **Asynchronous:** The result is delivered later via a **callback function, Promise, or Async/Await** construct.

Operation	Blocking (Synchronous)	Non-blocking (Asynchronous)
I/O	The CPU waits for the	The CPU handles other

	disk or network.	requests while waiting.
Code Flow	Sequential.	Executes tasks concurrently.

Example: Asynchronous File Read

The main thread executes `console.log('Start...')` and `console.log('End...')` immediately, while the file reading happens in the background. The callback runs only after the file is successfully read.

JavaScript

```
const fs = require('fs');

console.log('1. Start the file reading.');
```



```
// Asynchronous call - doesn't block the main thread
fs.readFile('data.txt', 'utf8', (err, data) => {
  if (err) throw err;
  console.log('3. File Content:', data); // This runs last
});
```



```
console.log('2. End of the script (Non-blocking).');
```

Explanation

This code demonstrates **asynchronous file reading** in Node.js using the built-in fs (file system) module. It shows that fs.readFile() doesn't block the main thread — Node.js continues running other code while the file is being read in the background.

Code Explanation

```
const fs = require('fs');
```

- Imports Node.js's built-in **File System (fs)** module.
- This module allows you to work with files — reading, writing, deleting, etc.

```
console.log('1. Start the file reading.');
```

- Prints a message to the console first.
- Just a log showing that the process of reading the file is starting.

```
fs.readFile('data.txt', 'utf8', (err, data) => {  
  if (err) throw err;  
  console.log('3. File Content:', data);  
});
```

- This reads the file **asynchronously**.
- 'data.txt' — the file to read.
- 'utf8' — ensures the file content is returned as text (not raw bytes).
- (err, data) => { ... } — **callback function** that runs **after** the file is read.

What happens inside the callback:

- If an error occurs (err is not null), it throws the error and stops execution.
- Otherwise, it prints the file's contents.

Important:

The callback runs **later**, after the file is finished reading — not immediately.

```
console.log('2. End of the script (Non-blocking).');
```

- This executes **immediately after line 2, before** the file reading is finished.

- That's because `fs.readFile()` is **non-blocking** — Node.js doesn't wait for it to finish.

Order of Execution

When you run this script, you'll see output like:

1. Start the file reading.
2. End of the script (Non-blocking).
3. File Content: <contents of data.txt>

Key Takeaways

Concept	Explanation
Asynchronous I/O	Node.js reads the file in the background so it doesn't freeze while waiting.
Non-blocking	Other code continues executing while <code>fs.readFile()</code> works.
Callback Function	Executes when the file operation completes (successfully or with an error).
Event Loop	Handles async operations like file reading, networking, etc., and runs callbacks when they're ready.

B. Single-Threaded Event Loop

Node.js uses a **single thread** for JavaScript execution, but it manages concurrency using the **Event Loop**—a continuously running mechanism that orchestrates non-blocking operations.

1. **Incoming Request:** A request arrives (e.g., an HTTP call).
2. **Execution:** The main thread processes the synchronous part of the code.

3. **I/O Offload:** When an I/O operation is encountered, it's handed off to the system kernel (or a thread pool), and the main thread moves on.
4. **Callback Queue:** Once the I/O operation completes, its associated **callback function** is placed in the Event Queue.
5. **Event Loop:** The loop checks the queue and pushes the callbacks onto the main thread for execution, ensuring the application remains responsive.

2. Modules and NPM (Node Package Manager)

A. The Module System (require and module.exports)

Node.js uses the **CommonJS** module system. Modules help organize code into reusable, encapsulated units.

Example: Creating and Exporting a Custom Module

File 1: calculator.js (The Module)

JavaScript

```
// Define functions
function add(a, b) {
  return a + b;
}
```

```
function subtract(a, b) {
```

```
    return a - b;
}

// Export the functions to be used outside this file
module.exports = {
  add: add,
  sub: subtract // Using a shorter name for the export
};
```

File 2: app.js (The Requiring File)

JavaScript

```
// Import the custom module
const calc = require('./calculator');

const sum = calc.add(10, 5);
const difference = calc.sub(10, 5);

console.log('Sum:', sum);          // Output: Sum: 15
console.log('Difference:', difference); // Output: Difference: 5
```

Explanation

1. Function Definitions

```
function add(a, b) {  
  return a + b;  
}
```

```
function subtract(a, b) {  
  return a - b;  
}
```

- `function add(a, b) →` defines a function named **add** that takes two parameters `a` and `b`, then returns their **sum**.
- `function subtract(a, b) →` defines another function named **subtract** that takes two parameters `a` and `b`, then returns their **difference** (`a - b`).

So:

```
add(5, 3);    // returns 8  
subtract(5, 3); // returns 2
```

2. Exporting Functions

```
module.exports = {  
  add: add,
```

```
sub: subtract // Using a shorter name for the export
};
```

- `module.exports` is a **Node.js feature** that allows you to make parts of your code available to other files.
- Here, you're exporting an **object** that contains two properties:
 - `add` → refers to the `add` function.
 - `sub` → refers to the `subtract` function, but given a **shorter alias** (`sub` instead of `subtract`).

So the exported module looks like:

```
{
  add: [Function: add],
  sub: [Function: subtract]
}
```

3. Using This Module in Another File

If this code is saved in a file called **calculator.js**, you can use it elsewhere like this:

```
// app.js
const calc = require('./calculator');

console.log(calc.add(10, 5)); // Output: 15
```

```
console.log(calc.sub(10, 5)); // Output: 5
```

Here:

- `require('./calculator')` imports the exported object.
- `calc.add()` calls the add function.
- `calc.sub()` calls the subtract function.

Summary

Section	Purpose
<code>function add(a, b)</code>	Adds two numbers
<code>function subtract(a, b)</code>	Subtracts two numbers
<code>module.exports</code>	Makes functions available to other files
<code>sub: subtract</code>	Gives a shorter alias when exporting

B. NPM (Node Package Manager)

NPM is the default package manager and the world's largest software registry. It simplifies managing and sharing packages (libraries).

- **package.json:** This file, created with `npm init`, holds metadata about your project, including its dependencies.
- **Installation:**
 - `npm install <package-name>`: Installs a package to the

node_modules folder and adds it to package.json.

- `npm install -g <package-name>`: Installs a package globally (e.g., command-line tools like nodemon).

1. Initialize Your Project

Before using npm, you need to create a **Node.js project folder**.

Steps:

Step 1: Create a new folder

```
mkdir myproject
```

```
cd myproject
```

Step 2: Initialize npm (creates package.json)

```
npm init
```

When you run npm init, you'll be prompted to fill details such as:

- project name
- version
- description
- entry point (like app.js)
- author, etc.

If you just want to skip the questions and use default settings:

```
npm init -y
```

This command creates a file called **package.json**, which looks like:

```
{  
  "name": "myproject",  
  "version": "1.0.0",  
  "main": "app.js",  
  "license": "ISC"  
}
```

2. Install Packages (Dependencies)

Example: Installing express

npm install express

This will:

Create a folder called **node_modules**

Add a dependency entry inside **package.json** like:

```
"dependencies": {  
  "express": "^4.18.2"  
}
```

You can then use it in your code:

```
const express = require('express');  
const app = express();
```

```
app.listen(3000, () => console.log('Server running on port 3000'));
```

3. Global Installation

Global installations are for tools you use across multiple projects (not inside one project folder).

For example:

```
npm install -g nodemon
```

This installs **nodemon** globally, allowing you to run:

```
nodemon app.js
```

to automatically restart your Node server when you save files.

4. Uninstall Packages

Local:

```
npm uninstall express
```

Global:

```
npm uninstall -g nodemon
```

5. View Installed Packages

npm list

npm list -g --depth=0 # For global packages

Summary Table

Command	Purpose
npm init	Creates package.json for your project
npm init -y	Creates package.json with defaults
npm install <package>	Installs locally & adds to dependencies
npm install -g <package>	Installs globally
npm uninstall <package>	Removes a package
npm list	Lists installed packages

3. Building a Simple Web Server

The **http core module** is fundamental for network communication and creating web servers.

Example: Basic HTTP Server

This server responds to any request with the status code 200 and the text "Hello Node.js Server!".

JavaScript

```
const http = require('http');

const hostname = '127.0.0.1'; // localhost
const port = 3000;

// The callback function handles every incoming request (req) and
// response (res)
const server = http.createServer((req, res) => {
  // Set the response status code and content type
  res.statusCode = 200;
```

```
res.setHeader('Content-Type', 'text/plain');

// Send the final response body and close the connection
res.end('Hello Node.js Server!\n');
});

// Start listening for connections
server.listen(port, hostname, () => {
  console.log(`Server running at http://${hostname}:${port}/`);
});

// To run: node server.js
```

Explanation

1. Import the HTTP Module

```
const http = require('http');
```

- Node.js has a built-in **http** module for creating web servers.
- `require('http')` imports it so you can use its functions (no installation needed).

2. Define Hostname and Port

```
const hostname = '127.0.0.1'; // localhost
```

```
const port = 3000;
```

- hostname is the **server address**.
127.0.0.1 refers to your own computer (localhost).
- port is the **entry point** number for the server — 3000 is common in development.

So the server will run at:

http://127.0.0.1:3000/

3. Create the Server

```
const server = http.createServer((req, res) => {  
  res.statusCode = 200;  
  res.setHeader('Content-Type', 'text/plain');  
  res.end('Hello Node.js Server!\n');  
});
```

What happens here:

- `http.createServer()` creates the server.
- It takes a **callback function** (`req, res`) which runs every time a client (like a browser) sends a request.
 - `req` (request): contains information about the incoming request.
 - `res` (response): is what you send back.

Inside the callback:

1. `res.statusCode = 200;`
→ Sets HTTP status **200 (OK)**.
2. `res.setHeader('Content-Type', 'text/plain');`
→ Tells the browser you're sending **plain text** (not HTML or JSON).
3. `res.end('Hello Node.js Server!\n');`
→ Sends the final message and closes the connection.

4. Start the Server

```
server.listen(port, hostname, () => {  
  console.log(`Server running at http://${hostname}:${port}/`);  
});
```

- `.listen()` makes the server **start listening** for incoming requests.
- The callback runs once the server is ready.
- The message confirms the server is running and shows the URL.

5. Running the Server

1. Save the code as **server.js**
2. Open your terminal in that folder.
3. Run:
4. `node server.js`

5. You'll see:
6. Server running at `http://127.0.0.1:3000/`
7. Open your browser and visit
`http://127.0.0.1:3000/`
You'll see:
8. Hello Node.js Server!

Summary Table

Line	Purpose
<code>require('http')</code>	Loads Node's HTTP module
<code>createServer()</code>	Creates an HTTP server
<code>(req, res)</code>	Handles each incoming request and response
<code>res.statusCode = 200</code>	Sets HTTP status to OK
<code>res.setHeader('Content-Type', 'text/plain')</code>	Sets response type
<code>res.end()</code>	Sends the response and closes it
<code>listen(port, hostname)</code>	Starts the server listening on the given port

Using Express.js (The De-facto Framework)

Express.js is the most popular minimal and flexible Node.js web application framework, simplifying routing, middleware, and request handling.

Example: Express Server with Routing

1. **Install:** npm install express
2. **Code:**

JavaScript

```
const express = require('express');  
const app = express();  
const port = 4000;
```

```
// Route 1: GET request to the homepage ('/')  
app.get('/', (req, res) => {  
  res.send('Welcome to the Express Home Page!');  
});
```

```
// Route 2: GET request to an API endpoint ('/api/data')  
app.get('/api/data', (req, res) => {
```

```
    const serverData = { message: "Data fetched successfully", timestamp:  
new Date() };  
    // res.json() sends the data as JSON  
    res.json(serverData);  
});
```

```
// Start the server  
app.listen(port, () => {  
    console.log(`Express server running on http://localhost:${port}`);  
});
```

```
// To run: node express_server.js
```